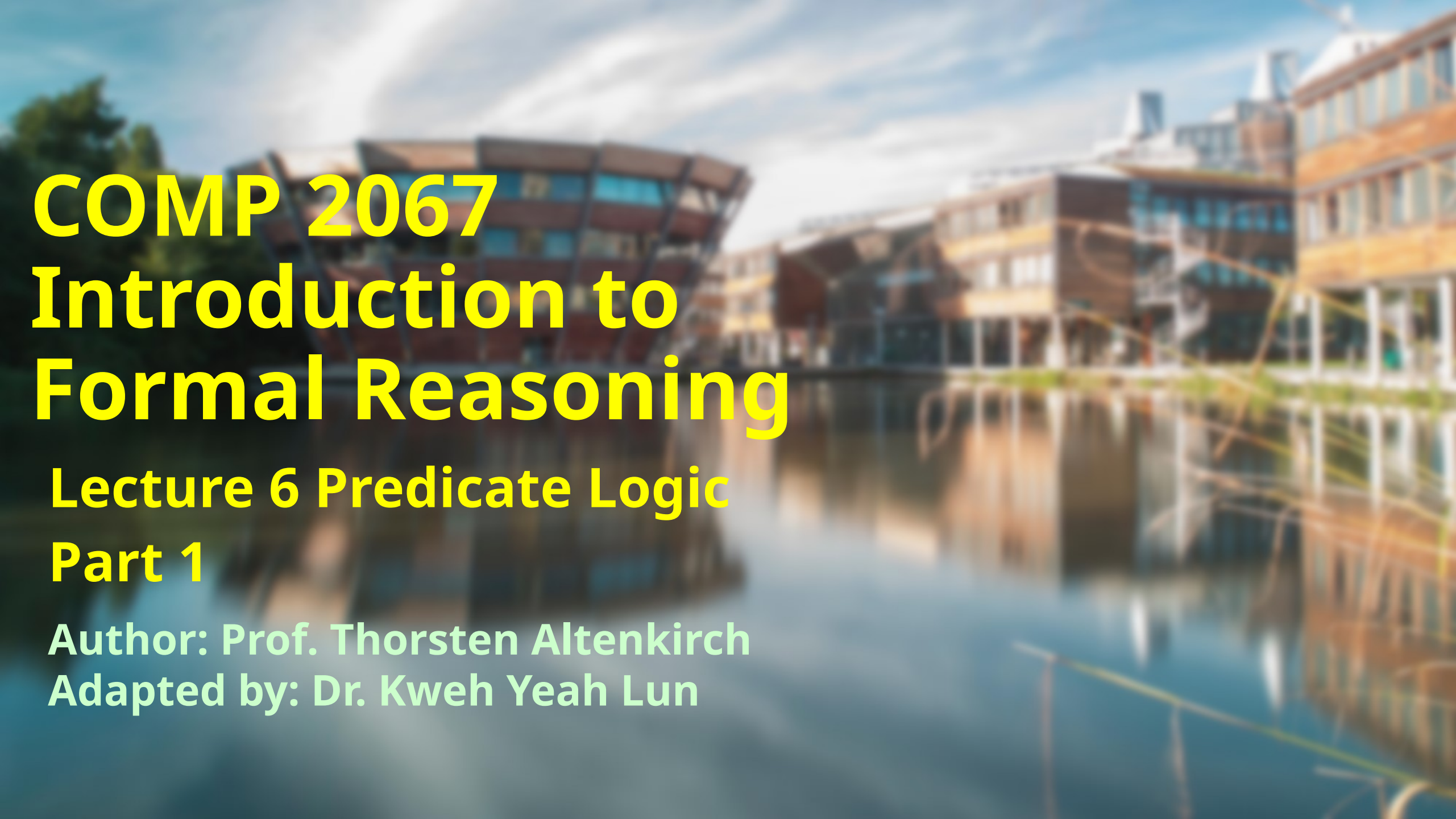

COMP 2067 Introduction to Formal Reasoning

Lecture 6 Predicate Logic

Part 1

Author: Prof. Thorsten Altenkirch
Adapted by: Dr. Kweh Yeah Lun

University of Nottingham
UK | CHINA | MALAYSIA

Learning Outcome

- At the end of this lecture, you will be able to
- explain the differences between proposition and predicate
- differentiate universal quantifier and existential quantifier
- dealing with universal quantifier and existential quantifier in Lean
- apply the tactics have and existsi

University of Nottingham
UK | CHINA | MALAYSIA

Difference between Predicate and Proposition

- Proposition
  - **a statement or assertion that can be either true or false**
  - have a **fixed truth value**
  - form the basis for logical reasoning and mathematical statements
  - **Example: 1 + 1 = 2, Kweh can fly**

- Predicate
  - **a function-like construct that takes one or more inputs (arguments) and returns a proposition**
  - the **truth value of a predicate depends on the input values**
    - when applied to specific objects, a predicate evaluates to a proposition, which can be true or false
  - Predicates are often used to define properties and conditions in mathematics and logic

University of Nottingham
UK | CHINA | MALAYSIA

Predicates, relations and quantifiers

- Predicate logic extends propositional logic
- It can be used to describe objects and their properties.
- The objects are organized in types, such as ℕ : Type the type of natural numbers **{0,1,2,3,...}** or bool : Type the type of booleans **{tt,ff},** etc.

- **Example: Prime : ℕ → Prop** express that a number is a prime number.
- Propositions: Prime 3 and Prime 4
- **Predicates may have several inputs** in which usually known as **relations**
- **Examples ≤ : ℕ → ℕ → Prop** form propositions like 2 ≤ 3.
- In this

```
variables PP QQ : A → Prop
```

edicates will be used.

Try it!

University of Nottingham
UK | CHINA | MALAYSIA

Quantifiers

- **Quantifiers** can be used to form new propositions:
  - **universal quantification (∀):**
    - ∀ x : A , PP x, means all x in A satisfy PP x.
  - **existential quantification (∃):**
    - ∃ x : A, PP x, means there is an x in A satisfying PP x.
- Both quantifiers bind weaker than any other propositional operator.
- Example: ∀ x : A, PP x ∧ Q means ∀ x : A , (PP x ∧ Q).
- Parentheses are needed to limit the scope.
- Example: (∀ x : A, PP x) ∧ Q which has a different meaning to the proposition before.

- Example: ∀ x:A , (∃ x : A , PP x) ∧ QQ x, here the x in PP x refers to ∃ x : A while the x in QQ x refers to ∀ x : A.
- Bound variables can be consistently renamed, hence the previous proposition is the same as ∀ y:A , (∃ z : A , PP z) ∧ QQ y, which is preferable since shadowing variables should be avoided because it confuses the human reader.

University of Nottingham
UK | CHINA | MALAYSIA

The universal quantifier

- To prove that a proposition of the form ∀ x : A , PP x holds, assume that there is an arbitrary element a in A and prove it for this generic element, i.e. to prove PP a, use assumption a to do this.
- For assumption h : ∀ x : A , PP x and the current goal is PP a for some a : A then use apply h to prove our goal.
- Some combination of implication and for all like h : ∀ x : A, PP x → QQ x and now if the current goal is QQ a and invoke apply h, Lean will instantiate x with a and the goal becomes PP a.

- Example: **(∀ x : A, PP x) → (∀ y : A, PP y → QQ y) → ∀ z : A , QQ z**
- Here is a possible translation into English: let's assume A stands for the type of students in the class, PP x means x is clever and QQ x means x is funny, then:
  - *If all students are clever then if all clever students are funny then all students are funny.*

- Try it!

```
example : (∀ x : A, PP x)
  → (∀ y : A, PP y → QQ y)
  → ∀ z : A , QQ z :=
begin
  assume p pq a,
  apply pq,
  apply p,
end
```

variables A B C : Type

- A, B, C: these are variables.
- : Type: This means that the variables A, B, and C are of type Type.
  - "a collection of all types", universe of types.
  - Examples of things that would have the type Type are:
    - Nat (natural numbers), Int (integers), Bool (booleans), List Nat (lists of natural numbers), A → B (functions from type A

- Example: Let's prove a logical equivalence involving ∀ and ∧, namely that they can be interchanged.

  (∀ x : A, PP x ∧ QQ x) ↔ (∀ x : A , PP x) ∧ (∀ x : A, QQ x)

- To illustrate this:
  - *All students are clever and funny is the same as saying that all students are clever and all students are funny.*

University of Nottingham
UK | CHINA | MALAYSIA

- Try it!

```
example : (∀ x : A, PP x ∧ QQ x)
  ↔ (∀ x : A , PP x) ∧ (∀ x : A, QQ x) :=
  begin
    constructor,
    assume h,
    constructor,
    assume a,
    have pq : PP a ∧ QQ a,
    apply h,
    cases pq with pa qa,
    exact pa,
    assume a,
    have pq : PP a ∧ QQ a,
    apply h,
    cases pq with pa qa,
    exact qa,
    assume h,
    cases h with hp hq,
    assume a,
    constructor,
    apply hp,
    apply hq,
  end
```

- **have**
  - The **have** tactic is used to **introduce a new hypothesis or intermediate result within a proof.**
  - It allows you to state and prove a subgoal separately before using it to continue the main proof.
  - particularly useful for breaking down complex proofs into smaller, more manageable steps.

- After assume a:

```
h : ∀ (x : A), PP x ∧ QQ x,
a : A
⊢ PP a
```

- Cannot apply h because PP a is not the conclusion of the assumption.
- The idea is that PP a ∧ QQ a can be proven from h and from this, prove PP a.
- Hence, have pq : PP a ∧ QQ a, which creates a new go

  but also inserts an assumption pq in the original goa

```
h : ∀ (x : A), PP x ∧ QQ x,
a : A
⊢ PP a ∧ QQ a

h : ∀ (x : A), PP x ∧ QQ x,
a : A,
pq : PP a ∧ QQ a
⊢ PP a
```

University of Nottingham
UK | CHINA | MALAYSIA

- Now can prove PP a ∧ QQ a using apply h and then left with:

```
h : ∀ (x : A), PP x ∧ QQ x,
a : A,
pq : PP a ∧ QQ a
⊢ PP a
```

- This can be done using cases on pq.

University of Nottingham
UK | CHINA | MALAYSIA

The existential quantifier

- **To prove a proposition of the form ∃ x : A , PP x, it is sufficient to prove PP a for any a : A.**
- Use **existsi a** for this and then prove PP a.
- Note that a can be any expression of type A, not necessarily a variable.
- On the other hand, to use an assumption of the form h : ∃ x : A, P x, use cases h with x px which replaces h with two assumptions x : A and px : P x.

- **Example:**

  (∃ x : A, PP x) → (∀ y : A, PP y → QQ y) → ∃ z : A , QQ z

- Here is the English version using the same translation as before:
  - *If there is a clever student and all clever students are funny, then there is a funny student.*

University of Nottingham
UK | CHINA | MALAYSIA

- Try it!

```
example : (∃ x : A, PP x)
  → (∀ y : A, PP y → QQ y)
  → ∃ z : A , QQ z :=
begin
  assume p pq,
  cases p with a pa,
  existsi a,
  apply pq,
  exact pa,
end
```

- **Example:**

  (∃ x : A, PP x ∨ QQ x) ↔ (∃ x : A , PP x) ∨ (∃ x : A, QQ x)

- Here is the English version:
  - *There is a student who is clever or funny is the same as saying there is a student who is funny or there is a student who is clever.*

University of Nottingham
UK | CHINA | MALAYSIA

- Try it!

```
example : (∃ x : A, PP x ∨ QQ x)
              ↔ (∃ x : A , PP x) ∨ (∃ x : A, QQ x) :=
begin
  constructor,
  assume h,
  cases h with a ha,
  cases ha with pa qa,
  left,
  existsi a,
  exact pa,
  right,
  existsi a,
  exact qa,
  assume h,
  cases h with hp hq,
  cases hp with a pa,
  existsi a,
  left,
  exact pa,
  cases hq with a qa,
  existsi a,
  right,
  exact qa,
end
```

University of Nottingham
UK | CHINA | MALAYSIA

- Give yourself a big clap.
- Survived once again? Great! ⏪
- Any question?

University of Nottingham
UK | CHINA | MALAYSIA

Next lecture

- Predicate Part 2…

University of Nottingham
UK | CHINA | MALAYSIA

"A language that doesn't have everything is actually easier to program in than some that do."

– Dennis Ritchie

Hope you all enjoy this lecture!